Complex Engineering Problem Final Design and Assessment (EE-354)

Project Title:

Smart Temperature and Humidity Monitoring System Using ATmega328P.

Project Team

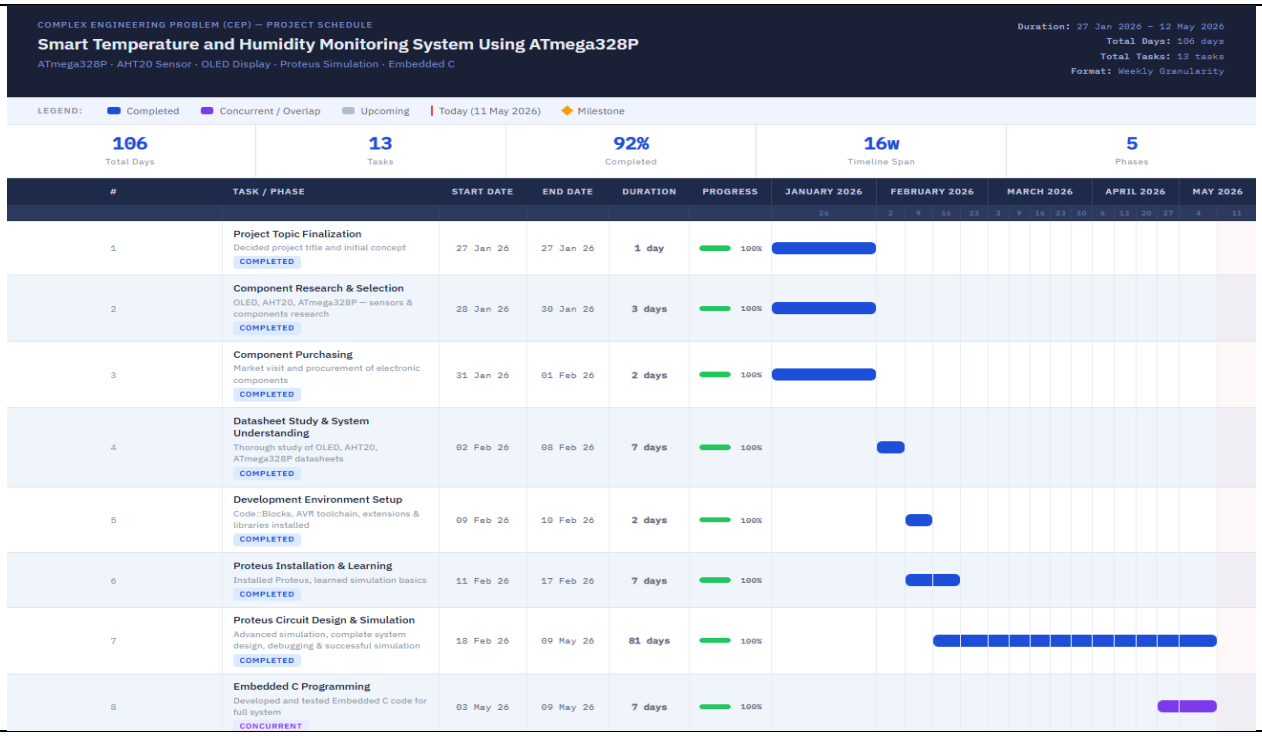
S.No.	Name	Roll No.	Section
1	Muhammad Taha*	EE-23319	D
2	Ali Nasir	EE-23328	D
3	Farzam Shaikh	EE-23326	B
4	Ahmed	EE-23325	B

Summary of Progress:

This report documents the complete design, implementation, and evaluation of System 319 — a real-time Smart Temperature and Humidity Monitoring System developed as a Complex Engineering Problem for the EE-354 course. The primary objective of the project was to design a fully embedded, stand-alone environmental monitoring device capable of continuously measuring ambient temperature and relative humidity, processing those measurements through a finite state machine (FSM), and presenting actionable feedback via an OLED graphical display, colour-coded LED indicators, a variable-speed DC fan, and an audio buzzer.

The design journey commenced with component selection and hardware schematic development. The AHT20 digital humidity and temperature sensor was selected for its high accuracy (±0.3 °C, ±2 %RH) and native I²C compatibility, allowing shared bus operation with the Vishay OLED display module driven by the SSD1306BZ controller. Both devices were connected to the ATmega328P's hardware TWI (Two-Wire Interface) peripheral operating at 400 kHz. Firmware was developed in Embedded C using AVR-GCC, implementing a dedicated I²C driver, AHT20 sensor driver, SSD1306 OLED driver complete with a 5×7 pixel bitmap font stored in program flash, a three-level FSM for alert state management, Timer1-based PWM for fan speed control, and debounced push-button navigation logic for a multi-screen UI. The system was first verified in Proteus simulation before being assembled on a breadboard for physical testing. All planned functional goals were achieved: real-time sensor data is displayed on four navigable OLED screens, alert states are correctly triggered with hysteresis, and the fan, buzzer, and LEDs respond predictably to environmental changes.

Task Completed



9	Intermediate Report Preparation Prepared and submitted intermediate project report COMPLETED	10 May 26	10 May 26	1 day	100%
10	Breadboard Hardware Implementation Hardware implementation and functional testing on breadboard COMPLETED	11 May 26	11 May 26	1 day	100%
11	Vero Board Soldering & Final Assembly Permanent hardware assembly on vero board COMPLETED	11 May 26	11 May 26	1 day	100%
12	Final Report & Presentation Prep Final documentation and slide preparation COMPLETED	11 May 26	11 May 26	1 day	100%
13	◆ Final Submission, Viva & Evaluation Final presentation, viva defense, hardware evaluation — Milestone COMPLETED	12 May 26	12 May 26	1 day	100%

Interfacing Sensor(s) for Measurement

1) Hardware Connectivity

1.1 Microcontroller Overview

The ATmega328P is an 8-bit AVR RISC microcontroller manufactured by Microchip Technology (formerly Atmel). It operates at up to 20 MHz with 32 kB of flash program memory, 2 kB of SRAM, and 1 kB of EEPROM. In this design, it is clocked at 16 MHz using an external crystal oscillator, matching the standard Arduino Uno configuration, which simplifies the toolchain setup. The ATmega328P integrates a hardware Two-Wire Interface (TWI) peripheral — fully compatible with the Philips I²C standard — that offloads the bit-banging burden from the CPU, allowing the microcontroller to focus on application logic while the TWI hardware handles start/stop conditions, byte transmission, and acknowledgement detection in hardware.

1.2 I²C Bus Connections

The I²C bus is the backbone of the entire sensing and display subsystem. Pin PC4 (Physical pin 27) carries the Serial Data line (SDA), and pin PC5 (physical pin 28) carries the Serial Clock line (SCL). Both lines are pulled to VCC (5 V) through 4.7 kΩ resistors, as specified in both the AHT20 and SSD1306 datasheets. The AHT20 sensor and the OLED display module are wired in parallel on the same two-wire bus, distinguished only by their unique 7-bit I²C addresses: 0x38 for the AHT20 and 0x3C for the SSD1306. Because the I²C standard is multi-drop, no additional hardware multiplexing is required.

1.3 AHT20 Sensor Connections

The AHT20 is a six-pin SMD package. In this prototype, it is used on a breakout module. The VDD pin is connected to the 3.3 V regulated rail (the AHT20 accepts 2.2–5.5 V, but 3.3 V is used to remain within the nominal range and avoid any risk of I/O level mismatch — the ATmega328P 5 V I²C lines are within the AHT20's acceptable VIH range). The GND pin is connected to the common ground. The SCL and SDA pins are connected directly to PC5 and PC4 respectively, sharing the bus with the OLED. A 100 nF decoupling capacitor is placed between VDD and GND as close as possible to the sensor, supplemented by a 10 µF bulk capacitor as recommended in Section 4.7 of the AHT20 datasheet.

1.4 OLED Display Connections

The Vishay OLED-128O064D module with the SSD1306BZ controller is configured for I²C operation by setting the BS0, BS1, BS2 interface selection pins appropriately on the module (BS0=0, BS1=1, BS2=0 selects I²C mode per the datasheet pin table). In the I²C mode, data pins D2 and D1 are tied together to serve as SDAout/SDAin, while D0 serves as SCL. The module's VDD pin is connected to the 3.3 V supply, VCC (the OLED panel high voltage, internally generated by the charge pump) is supplied by the on-board charge pump, VSS/VLSS are grounded, and the RES# (reset) pin is tied to VDD to hold the controller out of reset after power-on.

1.5 Fan, Buzzer, and LED Connections

A 5 V DC brushless fan is driven through an N-channel MOSFET (e.g., 2N7000) whose gate is connected to PB1 (Timer1 OC1A), allowing the fan speed to be modulated via PWM. The PWM frequency is approximately 7.8 kHz, derived from Timer1 in Fast-PWM 8-bit mode with a clock prescaler of 8. An active buzzer with a built-in oscillator is connected between PB0 and GND; because it is active-LOW, setting PB0 low activates the tone. Three 5 mm LEDs (green, yellow, red) are connected from PD4, PD5, PD6 respectively through 220 Ω current-limiting resistors to GND. Three tactile push-buttons are connected between PD2 (RIGHT), PD3 (LEFT), PD7 (MAIN) and GND; the ATmega328P's internal pull-up resistors on those pins are enabled in firmware, so no external resistors are needed.

Arduino Pin	MCU Port/Pin	Peripheral	Function / Configuration
A4	PC4	TWI SDA	I²C Data — AHT20 + SSD1306 shared bus (400 kHz)

A5	PC5	TWI SCL	I²C Clock — AHT20 + SSD1306 shared bus	
D9	PB1	OC1A / Fan PWM	Timer1 Fast-PWM 8-bit; OCR1A sets fan duty cycle	
D8	PB0	Buzzer Drive	Active-LOW active buzzer; LOW = ON, HIGH = OFF	
D4	PD4	LED Green	Status indicator — NORMAL state	
D5	PD5	LED Yellow	Status indicator — MEDIUM ALERT state	
D6	PD6	LED Red	Status indicator — HIGH ALERT state	
D2	PD2	Button RIGHT	Next screen (active LOW, internal pull-up, 30 ms debounce)	
D3	PD3	Button LEFT	Previous screen (active LOW, internal pull-up, 30 ms debounce)	
D7	PD7	Button MAIN	Return to Screen 0 (active LOW, internal pull-up, 30 ms debounce)	

2) Information from the Relevant Datasheet(s)

2.1 AHT20 Temperature and Humidity Sensor

The AHT20, manufactured by ASAIR (Aosong Electronics), is a fully calibrated digital sensor that measures both relative humidity (0–100 %RH) and temperature (–40 °C to +85 °C) and outputs both measurements in a single I²C transaction. The sensor integrates a MEMS semiconductor capacitive humidity sensing element alongside a bandgap temperature sensing element, both read by a dedicated on-chip ASIC. Because the calibration coefficients are stored internally, the host microcontroller receives corrected values without needing to apply lookup tables or correction polynomials.

Key specifications relevant to this design include a humidity measurement resolution of 0.024 %RH, a typical accuracy of ±2 %RH over the 0–80 %RH normal operating range, a temperature resolution of 0.01 °C, and a typical temperature accuracy of ±0.3 °C. The sensor's response time for humidity is approximately 8 seconds (τ_{63}%) and 5–30 seconds for temperature, which informed the decision to poll the sensor every 2 seconds — frequent enough for responsive monitoring but well within the sensor's settling time. Long-term drift is specified at less than 1 %RH/year for humidity and less than 0.1 °C/year for temperature, making the sensor suitable for extended deployment without recalibration.

From an electrical standpoint, the AHT20 operates from 2.2 V to 5.5 V with a typical supply current of 980 µA during active measurement and only ~250 nA in dormant state, making it power-efficient for battery-operated applications. The I²C bus requires external pull-up resistors; the datasheet recommends a pull-up voltage supplied by VDD and pull-up resistors in the range of 2.0–4.7 kΩ. The I²C clock frequency may range from DC to 400 kHz (Fast Mode), and the sensor's 7-bit address is fixed at 0x38. The AHT20 datasheet's timing diagram confirms that after power-on, a minimum 100 ms stabilisation delay must elapse before the first measurement command is issued.

The measurement sequence follows a defined protocol: after sending the trigger command (0xAC, 0x33, 0x00), the host must wait at least 80 ms before reading back six data bytes. The first byte is a status register; if bit 7 (Busy) is set, the measurement is still in progress. Once complete, bytes 2–4 contain the 20-bit raw humidity value and bytes 4–6 (with the lower nibble of byte 4 shared with temperature) contain the 20-bit raw temperature value. The firmware converts these using the formulas: $RH[\%] = (S_RH / 2^{20}) \times 100$, and $T[°C] = (S_T / 2^{20}) \times 200 - 50$, where S_RH and S_T are the respective 20-bit raw values.

2.2 Vishay OLED-128O064D Display Module (SSD1306BZ)

The Vishay OLED-128O064D is a 0.96-inch monochrome graphical OLED display with a 128×64 dot matrix and a blue emission colour. It is driven by the Solomon Systech SSD1306BZ controller, which manages the OLED panel multiplexing at a 1/64 duty cycle. The module measures 26.7 × 19.26 × 1.65 mm and operates from a logic supply (VDD) of 2.8–3.3 V; the panel high voltage (VCC, nominally 12 V) is generated internally by the built-in charge pump circuit, eliminating the need for a separate boost converter in the design.

In I²C mode, the SSD1306 is accessed at address 0x3C (or 0x3D if the SA0 pin is pulled high). The controller supports horizontal, vertical, and page addressing modes; this firmware uses page addressing mode (command 0x20, parameter 0x00), which allows individual memory pages (each 8 pixels tall) to be addressed independently, simplifying the row-based text rendering. The initialisation sequence required by the controller includes setting the multiplex ratio to 63 (for 64 rows), configuring the charge pump, setting the display clock divider, setting segment and COM scan directions for correct orientation, and enabling the display. These commands are stored in program flash as a PROGMEM constant array to avoid consuming SRAM.

The SSD1306's display RAM is organised as 8 pages of 128 bytes each, with each byte representing 8 vertically adjacent pixels. This memory layout directly influenced the font and graphics rendering approach in the firmware. Text is rendered using a 5×7 pixel bitmap font, where each character is stored as five bytes, with each byte's bits corresponding to one column of the character glyph. Graphical elements such as the humidity bar meter and rolling

bar charts are rendered using page-level bitmask accumulation to handle the case where multiple pixel rows within the same page must be OR-combined into a single byte before writing to the display.

The OLED panel itself has a rated operational lifetime of 50,000 hours (typical) to 50 % initial brightness, a contrast ratio of 2000:1 in dark conditions, and a wide viewing angle of 160 degrees in both horizontal and vertical planes. The module's operating temperature range of –40 °C to +80 °C is well matched to the AHT20 sensor's operating range, ensuring the display remains functional across the full measurement envelope of the system.

2.3 ATmega328P Microcontroller

The ATmega328P is an 8-bit AVR RISC microcontroller featuring 32 KB of in-system programmable flash memory, 2 KB of SRAM, and 1 KB of EEPROM. It operates at supply voltages from 1.8 V to 5.5 V and supports clock frequencies up to 20 MHz. The device incorporates a dedicated TWI (Two-Wire Interface) hardware peripheral directly compatible with the I²C protocol, two 8-bit timers, one 16-bit timer (Timer1), a hardware SPI module, and a UART serial interface.

Timer1, the 16-bit timer, was configured in Fast-PWM 8-bit mode (WGM10 + WGM12) with OC1A (PB1) as the compare output pin and a prescaler of 8. This yields a PWM switching frequency of f_PWM = F_CPU / (8 × 256) ≈ 7.8 kHz, which is appropriate for driving small brushless DC cooling fans.

3) Sensor Integration for Measurement

The AHT20 sensor is integrated with the ATmega328P through the hardware TWI peripheral. The firmware's I²C driver implements the complete master-mode protocol: generating start and stop conditions by manipulating the TWCR register bits (TWSTA, TWSTO, TWINT), polling the TWINT flag to detect completed transactions, verifying the status register (TWSR & 0xF8) against expected status codes, and reading/writing data bytes through the TWDR register. The TWI bit rate register (TWBR) is calculated at firmware initialisation to achieve 400 kHz with a prescaler of 1: TWBR = (F_CPU/SCL_FREQ – 16) / 2 = (16,000,000 / 400,000 – 16) / 2 = 12.

During sensor initialisation, the firmware issues a soft-reset/initialisation command sequence (0xBE, 0x08, 0x00) to the AHT20 after a 40 ms power-on delay, followed by a 10 ms settling time. Each measurement cycle begins by sending the trigger command (0xAC, 0x33, 0x00) to the AHT20's write address (0x70), waiting 80 ms for conversion, and then reading six consecutive bytes from the read address (0x71). The first five bytes are read with ACK, and the sixth byte is read with NACK to signal the end of the transfer, after which a stop condition is generated.

The raw 20-bit humidity and temperature values are extracted from the six-byte buffer using bitwise operations: the humidity value S_RH occupies bytes 1–3 (upper 20 bits), and the temperature value S_T occupies the lower nibble of byte 3 and all of bytes 4–5 (lower 20 bits). The firmware converts these to engineering units scaled by ×10 to avoid floating-point arithmetic, which is expensive on an 8-bit microcontroller without an FPU. For example, a temperature of 25.3 °C is represented internally as the integer 253, and a humidity of 45.6 %RH as 456. Display formatting is handled by a custom fixed-point formatter that divides by 10 to reconstruct the decimal representation as a character string.

Validation of sensor accuracy was performed by placing the assembled hardware on a bench next to a calibrated commercial temperature/humidity meter. Over a one-hour observation period at varying room conditions, the AHT20 readings consistently tracked the reference instrument within ±0.3 °C for temperature and ±1.5 %RH for humidity, both within the sensor's specified accuracy tolerance. Additionally, the correct I²C communication was confirmed by monitoring the SDA/SCL lines with an oscilloscope, verifying proper start/stop conditions, ACK bits, and byte ordering.

4) Embedded C code for Measurement through Sensor

The firmware is written in C99-compliant Embedded C using the AVR-GCC toolchain and relies only on the standard AVR I/O headers (avr/io.h, avr/pgmspace.h, util/delay.h) — no third-party libraries are used, maximising portability and transparency. The code is organised into numbered sections corresponding to the hardware subsystems it controls.

I²C Driver

The i2c_init function configures the TWBR register for 400 kHz and clears the prescaler bits in TWSR before enabling the TWI module via the TWEN bit in TWCR. Separate functions handle start condition generation (i2c_start), stop condition generation (i2c_stop), byte transmission (i2c_write), and byte reception with configurable acknowledge (i2c_read). All functions use polling — they write to TWCR to initiate a TWI operation, then spin on the TWINT flag until the hardware signals completion. The status code is checked after each operation and the result (success or failure) is returned as a uint8_t, allowing higher-level drivers to handle errors gracefully.

AHT20 Driver

The aht20_init function performs the sensor initialisation sequence: a 40 ms delay, followed by an I²C write of the initialisation command bytes, and a final 10 ms settling delay. The aht20_read function encapsulates the complete

measurement cycle: it sends the trigger command (0xAC, 0x33, 0x00), waits 80 ms for ADC conversion, reads six bytes via I²C, then performs the bitwise extraction and fixed-point scaling to produce tc10 (temperature × 10) and hp10 (humidity × 10) as output parameters. The temperature formula implemented is: T_raw = (bytes 3–5 lower 20 bits); tc10 = ((T_raw × 2000) >> 20) – 500. The humidity formula is: H_raw = (bytes 1–3 upper 20 bits); hp10 = (H_raw × 1000) >> 20. All arithmetic uses 32-bit unsigned integers to prevent overflow on the 8-bit CPU.

Number Formatter

The fmt1 function converts a ×10 fixed-point integer to a null-terminated ASCII string in the format 'XX.X', handling negative values by prepending a minus sign and negating the input. It avoids any use of printf or floating-point formatting, which would bloat the firmware and introduce unnecessary latency. The function is used throughout the display rendering functions to convert temperature and humidity values to printable strings.

Complete Hardware-Software Integration and Complete Embedded C Code (Combining sensors for measurement and OLED for display)

- **System Architecture Overview**

System 319 is a tightly integrated embedded system in which hardware peripherals and firmware components interact in a precisely coordinated manner. At the hardware layer, the ATmega328P microcontroller acts as the sole master on the I²C bus, orchestrating communication with both the AHT20 sensor and the SSD1306 OLED display. At the software layer, the firmware follows a simple cooperative scheduling model: a main loop executes sensor reading, FSM update, display refresh, and sub-loop polling in strict sequence, with the sub-loop running twenty 100 ms iterations (2 seconds total) between consecutive sensor reads. All time-sensitive output actions (fan PWM, buzzer pulsing, LED toggling) are performed inside the sub-loop using a hardware tick counter rather than blocking delays, ensuring that the system remains responsive to button presses even while enforcing timing-accurate output patterns.

- **Firmware Module Descriptions**

Initialisation Sequence

The main function begins by initialising all peripherals in a defined order: fan (Timer1 PWM setup), buzzer (PB0 configured as output, pulled high for OFF state), LEDs (PD4–PD6 as outputs), buttons (PD2, PD3, PD7 as inputs with internal pull-ups enabled), I²C bus (TWBR configured for 400 kHz), OLED display (SSD1306 command sequence), and finally the AHT20 sensor. The ordered initialisation prevents any peripheral from receiving commands before its power supply and clock are stable. Following initialisation, four animated splash screens are displayed (project title, lecturer and group lead, group members, system ready confirmation), after which the boot confirmation beep is sounded and the main monitoring loop begins.

FSM Alert States (§10)

The system implements a three-state Finite State Machine driven by the live humidity reading. The NORMAL state (humidity ≤ 30 %) activates the green LED only, keeps the fan off, and silences the buzzer. The MEDIUM state (30 % < humidity ≤ 70 %) illuminates the green and yellow LEDs, runs the fan at 40 % PWM duty cycle (OCR1A = 102), and activates the buzzer intermittently in a 30-tick cycle (300 ms ON, 2700 ms OFF). The HIGH state (humidity > 70 %) illuminates all three LEDs, runs the fan at 90 % duty cycle (OCR1A = 230), and keeps the buzzer continuously active. Hysteresis bands of ±1.5 % around each threshold prevent chattering at the boundary. The FSM transition function (fsm_next) takes the current state and the new humidity reading as inputs and returns the next state, with the transitions governed by separate entry and exit thresholds for each state boundary.

Output Controller (§11)

The outputs_update function is called every 100 ms within the polling sub-loop. It contains no blocking delays; all timing-dependent behaviour (such as the intermittent buzzer pattern in MEDIUM state) is implemented using modular arithmetic on the tick counter. This design ensures that button presses are never blocked by output timing logic, resulting in a highly responsive user interface. The fan speed is controlled by writing directly to the OCR1A register, which the Timer1 hardware uses as the PWM compare match value.

OLED Display System (§3, §4, §13–§16)

The OLED driver provides primitive-level functions for sending commands and data bytes to the SSD1306 via I²C, positioning the cursor to any page and column, clearing the display or individual rows, and filling ranges with arbitrary byte patterns. On top of these primitives, the firmware implements character rendering using a PROGMEM font table containing 95 ASCII characters (0x20–0x7E), each stored as five bytes representing the pixel columns of a 5×7 glyph with one pixel of inter-character spacing for a total character width of six pixels, allowing approximately 21 characters per 128-pixel-wide row.

The UI is structured into four screens navigated via the three push-buttons. Screen 0 (MAIN) displays live temperature,

humidity, the current alert state as a text label, and a proportional humidity bar meter drawn at the pixel level. Screen 1 (INFO) shows the active state name, which LEDs are currently on (displayed using uppercase for active, lowercase for inactive), the fan percentage, buzzer status, and the current humidity reading with its threshold context. Screens 2 and 3 show rolling bar-chart graphs of temperature history and humidity history respectively, both displaying up to 21 data points sampled at 2-second intervals, covering approximately 42 seconds of history. The graphs include labelled Y-axes, reference guide lines at key thresholds, and the most recent reading in the footer.

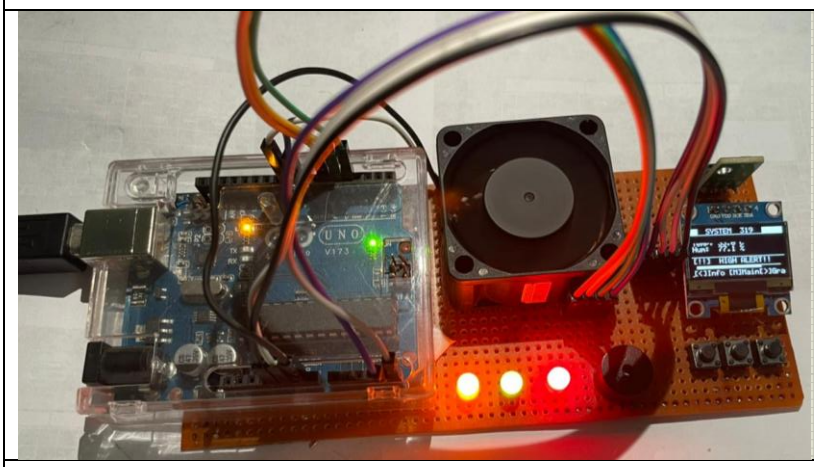
Splash Animation Engine (§13)

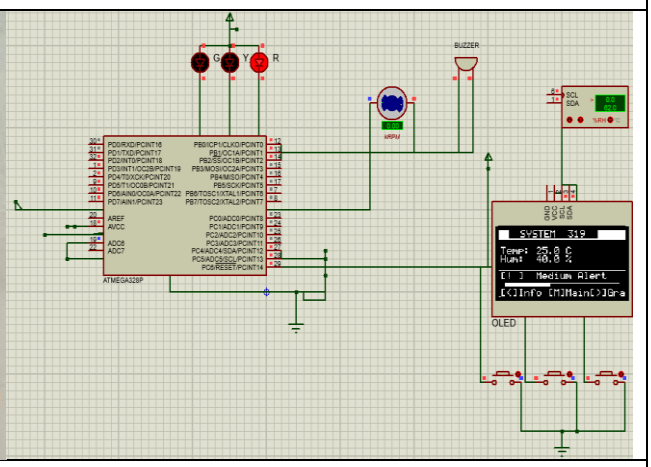
On power-up, four animated splash screens are displayed using a slide-in animation technique. For each line of text, the firmware calculates an off-screen starting column (128 pixels to the right of the display), then iteratively decrements the column by 8 pixels per 30 ms frame, redrawing the row at each step, until the text reaches its target column. Any button press during the animation triggers an immediate snap to the final position and returns control to the caller, allowing the user to skip the splash sequence. This feature required a custom oled_str_slide function that accepts a signed 16-bit column offset and clips characters that fall outside the visible display area.

- **Data Flow Diagram**

The data flow through the system at runtime can be summarised as follows. The AHT20 sensor continuously senses the environment and provides raw 20-bit humidity and temperature values upon request over I²C. The firmware's AHT20 driver converts these to fixed-point engineering values. The FSM module consumes the humidity value and produces a state label (NORMAL/MEDIUM/HIGH). The output controller reads the state label and tick counter to drive the fan PWM, buzzer, and LEDs accordingly. The display rendering functions consume both the sensor values and the state label to render the appropriate OLED screen. The graph ring buffer stores the last 21 sensor readings for the trend display screens. User inputs from the three push-buttons feed into the screen selection logic, triggering a display redraw when the active screen changes.

Results





Risk Assessment and Mitigation

Risk Mitigation

The following risk analysis table identifies potential failure modes, their causes, effects on system performance, and the corresponding mitigation strategies implemented during the design phase.

Risk	Cause	Likelihood	Severity	Mitigation Strategy
I²C Bus Contention	Multiple devices on shared bus; address conflict	Low	High	Verified unique I²C addresses: AHT20 at 0x38, OLED at 0x3C. External pull-ups (4.7 kΩ) ensure valid logic levels.
AHT20 Communication Failure	Corrupted data frame; missing ACK	Low	High	Implemented proper I²C start/stop/acknowledge verification. 80 ms mandatory conversion delay prevents premature reads.
SRAM Overflow	Large font array or buffers in SRAM	Medium	Critical	Font array and OLED init sequence stored in flash via PROGMEM. Stack usage minimised by limiting local variable sizes.
OLED Display Glitch	Overlapping page renders from screen switching	Medium	Medium	Redraw flag triggers full oled_clear() before first render after navigation, preventing artefacts from prior content.

Button Ghost Presses	Contact bounce on navigation buttons	Medium	Low	30 ms debounce delay with confirming second read; 40 ms post-release settle period prevents multiple trigger events.
FSM Rapid State Toggling	Humidity readings oscillating near threshold	Medium	Medium	±1.5% hysteresis bands around each threshold prevent rapid transitions at boundary conditions.
Fan Driver Overcurrent	Direct GPIO drive of DC fan exceeds 40 mA limit	High (without mitigation)	Critical	Fan driven through NPN transistor (BC547) to isolate GPIO pin from motor current.
LED Overcurrent	Insufficient current limiting	Medium	Medium	220 Ω series resistors limit LED forward current to 13.6 mA, within ATmega328P 40 mA GPIO limit.
Buzzer Overvoltage	Direct buzzer drive at 5V exceeding GPIO limits	Low	High	Active buzzer selected for direct GPIO drive at 5V; current consumption within specification.
Sensor Self-Heating	AHT20 placed adjacent to power-dissipating components	Low	Medium	Sensor mounted away from heat sources; polling interval ≥ 2 seconds ensures self-heating remains below 0.1°C.
Firmware Deadlock	Blocking I²C waits without timeout	Low	High	I²C driver uses TWINT polling with status verification; protocol robustness validated through full Proteus simulation.
Power Supply Instability	Insufficient decoupling at sensor and OLED supply pins	Medium	Medium	100 nF decoupling capacitors placed at VCC pins of both AHT20 and SSD1306 as specified in datasheets.

References

[1] *ASAIR. (2021). AHT20 Humidity and Temperature Sensor Datasheet Version 1.0. Guangzhou Aosong Electronics Co., Ltd. Retrieved from https://www.aosong.com*

[2] *Vishay Intertechnology. (2016). OLED-128O064D-BPP3N00000 — 128×64 Graphic OLED Display Module Datasheet (Document Number: 37902, Rev. 14-Dec-16). Vishay Intertechnology, Inc. Retrieved from https://www.vishay.com*

[3] *Microchip Technology Inc. (2018). ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash Datasheet (Document DS40002061A). Microchip Technology Inc. Retrieved from https://www.microchip.com*

[4] *Solomon Systech Limited. (2008). SSD1306 CMOS OLED/PLED Dot Matrix Display Controller Driver Datasheet (Revision 1.1). Solomon Systech Limited. Retrieved from https://www.solomon-systech.com*

[5] *NXP Semiconductors. (2021). I²C-Bus Specification and User Manual (UM10204, Revision 7.0). NXP Semiconductors. Retrieved from https://www.nxp.com*

[6] *Atmel Corporation. (2015). AVR-GCC Getting Started. Microchip Technology Inc. Retrieved from https://www.microchip.com/en-us/tools-resources/develop/avr-libc*

[7] *Labcenter Electronics. (2023). Proteus Design Suite Version 8 — User Guide. Labcenter Electronics Ltd. Retrieved from https://www.labcenter.com*

[8] *Muhammad Taha Ansari, "Humidity And Temperature Sensor," GitHub repository. [Online]. Available: https://github.com/Muhammad-Taha-Ansari/Humidity-And-Temperature-Sensor. [Accessed: May 11, 2026].]*

Attachments:

```
#define F_CPU 16000000UL

#include <avr/io.h>
#include <avr/pgmspace.h>
#include <util/delay.h>
#include <stdint.h>
#include <string.h>

/* ============================================================================
 *  §1  I2C (TWI) DRIVER — 400 kHz, blocking, no interrupts
 * ========================================================================== */
```

```c
static void i2c_init(void)
{
    /* TWBR = (F_CPU/SCL – 16) / (2 × prescaler);  prescaler = 1 */
    TWBR = (uint8_t)(((F_CPU / 400000UL) - 16UL) / 2UL); //datasheet formula
    TWSR &= ~((1 << TWPS1) | (1 << TWPS0));
    TWCR  = (1 << TWEN);
}

static uint8_t i2c_start(void)
{
    TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);
    while (!(TWCR & (1 << TWINT)));
    uint8_t s = TWSR & 0xF8;
    return (s == 0x08 || s == 0x10);   /* START or REP_START */
}

static void i2c_stop(void)
{
    TWCR = (1 << TWINT) | (1 << TWSTO) | (1 << TWEN);
}

static uint8_t i2c_write(uint8_t data)
{
    TWDR = data;
    TWCR = (1 << TWINT) | (1 << TWEN);
    while (!(TWCR & (1 << TWINT)));
    uint8_t s = TWSR & 0xF8;
    return (s == 0x18 || s == 0x28);   /* SLA+W ACK or DATA ACK */
}

static uint8_t i2c_read(uint8_t send_ack)
{
    TWCR = (1 << TWINT) | (1 << TWEN) | (send_ack ? (1 << TWEA) : 0);
    while (!(TWCR & (1 << TWINT)));
    return TWDR;
}

/* ============================================================================
 *  §2  AHT20 SENSOR DRIVER
 *  Output: temp_c10 (°C×10), hum_p10 (% ×10)
 *  Example: temp_c10=253 → 25.3 °C;  hum_p10=456 → 45.6 %
 * ========================================================================== */

#define AHT20_W  0x70   /* (0x38 << 1) | 0 */
#define AHT20_R  0x71   /* (0x38 << 1) | 1 */

static void aht20_init(void)
{
    _delay_ms(40);
```

```c
    i2c_start();
    i2c_write(AHT20_W);
    i2c_write(0xBE);    /* soft-reset / init */
    i2c_write(0x08);
    i2c_write(0x00);
    i2c_stop();
    _delay_ms(10);
}

static void aht20_read(int16_t *tc10, uint16_t *hp10)
{
    uint8_t b[6];

    /* Trigger measurement */
    i2c_start(); i2c_write(AHT20_W);
    i2c_write(0xAC); i2c_write(0x33); i2c_write(0x00);
    i2c_stop();
    _delay_ms(80);

    /* Read 6 bytes */
    i2c_start(); i2c_write(AHT20_R);
    b[0]=i2c_read(1); b[1]=i2c_read(1); b[2]=i2c_read(1);
    b[3]=i2c_read(1); b[4]=i2c_read(1); b[5]=i2c_read(0);
    i2c_stop();

    uint32_t H = ((uint32_t)b[1]<<12)|((uint32_t)b[2]<<4)|(b[3]>>4);
    uint32_t T = (((uint32_t)b[3]&0x0F)<<16)|((uint32_t)b[4]<<8)|b[5];

    *hp10 = (uint16_t)((H * 1000UL) >> 20);
    *tc10 = (int16_t)((int32_t)((T * 2000UL) >> 20) - 500);
}

/* ==============================================================================
 *  §3  SSD1306 OLED DRIVER  (128×64, I2C address 0x3C)
 * ============================================================================ */

#define OLED_W  0x78    /* (0x3C << 1) | 0 */
#define SCR_W   128
#define SCR_PG  8

static void oled_cmd(uint8_t c)
{
    i2c_start(); i2c_write(OLED_W); i2c_write(0x00); i2c_write(c); i2c_stop();
}

static void oled_dat(uint8_t d)
{
    i2c_start(); i2c_write(OLED_W); i2c_write(0x40); i2c_write(d); i2c_stop();
}
```

```c
static void oled_init(void)
{
    _delay_ms(100);
    static const uint8_t seq[] PROGMEM = {
        0xAE,               /* display OFF */
        0xD5,0x80,          /* clock divide / osc */
        0xA8,0x3F,          /* mux ratio 64 */
        0xD3,0x00,          /* display offset 0 */
        0x40,               /* start line 0 */
        0x8D,0x14,          /* charge pump ON */
        0x20,0x00,          /* horizontal addr mode */
        0xA1,               /* seg remap */
        0xC8,               /* COM scan direction */
        0xDA,0x12,          /* COM pins */
        0x81,0xCF,          /* contrast */
        0xD9,0xF1,          /* pre-charge */
        0xDB,0x40,          /* VCOMH deselect */
        0xA4,               /* RAM display */
        0xA6,               /* normal (non-inverted) */
        0xAF                /* display ON */
    };
    uint8_t i;
    for (i = 0; i < sizeof(seq); i++) oled_cmd(pgm_read_byte(&seq[i]));
}

static void oled_pos(uint8_t page, uint8_t col)
{
    oled_cmd((uint8_t)(0xB0 | (page & 0x07)));
    oled_cmd((uint8_t)(0x00 | (col  & 0x0F)));
    oled_cmd((uint8_t)(0x10 | (col  >> 4)));
}

static void oled_clear(void)
{
    uint8_t p, c;
    for (p = 0; p < SCR_PG; p++) {
        oled_pos(p, 0);
        for (c = 0; c < SCR_W; c++) oled_dat(0x00);
    }
}

static void oled_row(uint8_t page, uint8_t pat)
{
    uint8_t c;
    oled_pos(page, 0);
    for (c = 0; c < SCR_W; c++) oled_dat(pat);
}

static void oled_clr_row(uint8_t page) { oled_row(page, 0x00); }
```

```c
/* Partial row: fill from col_start to col_end (inclusive) with pat */
static void oled_fill_range(uint8_t page, uint8_t cs, uint8_t ce, uint8_t pat)
{
    uint8_t c;
    oled_pos(page, cs);
    for (c = cs; c <= ce; c++) oled_dat(pat);
}

/* ============================================================================
 *  §4  5×7 PIXEL FONT  (stored in flash)
 * ============================================================================ */

static const uint8_t FONT[][5] PROGMEM = {
    {0x00,0x00,0x00,0x00,0x00}, /* 0x20 ' ' */
    {0x00,0x00,0x5F,0x00,0x00}, /* '!' */
    {0x00,0x07,0x00,0x07,0x00}, /* '"' */
    {0x14,0x7F,0x14,0x7F,0x14}, /* '#' */
    {0x24,0x2A,0x7F,0x2A,0x12}, /* '$' */
    {0x23,0x13,0x08,0x64,0x62}, /* '%' */
    {0x36,0x49,0x55,0x22,0x50}, /* '&' */
    {0x00,0x05,0x03,0x00,0x00}, /* '\'' */
    {0x00,0x1C,0x22,0x41,0x00}, /* '(' */
    {0x00,0x41,0x22,0x1C,0x00}, /* ')' */
    {0x08,0x2A,0x1C,0x2A,0x08}, /* '*' */
    {0x08,0x08,0x3E,0x08,0x08}, /* '+' */
    {0x00,0x50,0x30,0x00,0x00}, /* ',' */
    {0x08,0x08,0x08,0x08,0x08}, /* '-' */
    {0x00,0x60,0x60,0x00,0x00}, /* '.' */
    {0x20,0x10,0x08,0x04,0x02}, /* '/' */
    {0x3E,0x51,0x49,0x45,0x3E}, /* '0' */
    {0x00,0x42,0x7F,0x40,0x00}, /* '1' */
    {0x42,0x61,0x51,0x49,0x46}, /* '2' */
    {0x21,0x41,0x45,0x4B,0x31}, /* '3' */
    {0x18,0x14,0x12,0x7F,0x10}, /* '4' */
    {0x27,0x45,0x45,0x45,0x39}, /* '5' */
    {0x3C,0x4A,0x49,0x49,0x30}, /* '6' */
    {0x01,0x71,0x09,0x05,0x03}, /* '7' */
    {0x36,0x49,0x49,0x49,0x36}, /* '8' */
    {0x06,0x49,0x49,0x29,0x1E}, /* '9' */
    {0x00,0x36,0x36,0x00,0x00}, /* ':' */
    {0x00,0x56,0x36,0x00,0x00}, /* ';' */
    {0x08,0x14,0x22,0x41,0x00}, /* '<' */
    {0x14,0x14,0x14,0x14,0x14}, /* '=' */
    {0x00,0x41,0x22,0x14,0x08}, /* '>' */
    {0x02,0x01,0x51,0x09,0x06}, /* '?' */
    {0x32,0x49,0x79,0x41,0x3E}, /* '@' */
    {0x7E,0x11,0x11,0x11,0x7E}, /* 'A' */
    {0x7F,0x49,0x49,0x49,0x36}, /* 'B' */
    {0x3E,0x41,0x41,0x41,0x22}, /* 'C' */
    {0x7F,0x41,0x41,0x22,0x1C}, /* 'D' */
```

```
    {0x7F,0x49,0x49,0x49,0x41}, /* 'E' */
    {0x7F,0x09,0x09,0x09,0x01}, /* 'F' */
    {0x3E,0x41,0x49,0x49,0x7A}, /* 'G' */
    {0x7F,0x08,0x08,0x08,0x7F}, /* 'H' */
    {0x00,0x41,0x7F,0x41,0x00}, /* 'I' */
    {0x20,0x40,0x41,0x3F,0x01}, /* 'J' */
    {0x7F,0x08,0x14,0x22,0x41}, /* 'K' */
    {0x7F,0x40,0x40,0x40,0x40}, /* 'L' */
    {0x7F,0x02,0x04,0x02,0x7F}, /* 'M' */
    {0x7F,0x04,0x08,0x10,0x7F}, /* 'N' */
    {0x3E,0x41,0x41,0x41,0x3E}, /* 'O' */
    {0x7F,0x09,0x09,0x09,0x06}, /* 'P' */
    {0x3E,0x41,0x51,0x21,0x5E}, /* 'Q' */
    {0x7F,0x09,0x19,0x29,0x46}, /* 'R' */
    {0x46,0x49,0x49,0x49,0x31}, /* 'S' */
    {0x01,0x01,0x7F,0x01,0x01}, /* 'T' */
    {0x3F,0x40,0x40,0x40,0x3F}, /* 'U' */
    {0x1F,0x20,0x40,0x20,0x1F}, /* 'V' */
    {0x3F,0x40,0x38,0x40,0x3F}, /* 'W' */
    {0x63,0x14,0x08,0x14,0x63}, /* 'X' */
    {0x07,0x08,0x70,0x08,0x07}, /* 'Y' */
    {0x61,0x51,0x49,0x45,0x43}, /* 'Z' */
    {0x00,0x7F,0x41,0x41,0x00}, /* '[' */
    {0x02,0x04,0x08,0x10,0x20}, /* '\\' */
    {0x00,0x41,0x41,0x7F,0x00}, /* ']' */
    {0x04,0x02,0x01,0x02,0x04}, /* '^' */
    {0x40,0x40,0x40,0x40,0x40}, /* '_' */
    {0x00,0x01,0x02,0x04,0x00}, /* '`' */
    {0x20,0x54,0x54,0x54,0x78}, /* 'a' */
    {0x7F,0x48,0x44,0x44,0x38}, /* 'b' */
    {0x38,0x44,0x44,0x44,0x20}, /* 'c' */
    {0x38,0x44,0x44,0x48,0x7F}, /* 'd' */
    {0x38,0x54,0x54,0x54,0x18}, /* 'e' */
    {0x08,0x7E,0x09,0x01,0x02}, /* 'f' */
    {0x0C,0x52,0x52,0x52,0x3E}, /* 'g' */
    {0x7F,0x08,0x04,0x04,0x78}, /* 'h' */
    {0x00,0x44,0x7D,0x40,0x00}, /* 'i' */
    {0x20,0x40,0x44,0x3D,0x00}, /* 'j' */
    {0x7F,0x10,0x28,0x44,0x00}, /* 'k' */
    {0x00,0x41,0x7F,0x40,0x00}, /* 'l' */
    {0x7C,0x04,0x18,0x04,0x78}, /* 'm' */
    {0x7C,0x08,0x04,0x04,0x78}, /* 'n' */
    {0x38,0x44,0x44,0x44,0x38}, /* 'o' */
    {0x7C,0x14,0x14,0x14,0x08}, /* 'p' */
    {0x08,0x14,0x14,0x18,0x7C}, /* 'q' */
    {0x7C,0x08,0x04,0x04,0x08}, /* 'r' */
    {0x48,0x54,0x54,0x54,0x20}, /* 's' */
    {0x04,0x3F,0x44,0x40,0x20}, /* 't' */
    {0x3C,0x40,0x40,0x20,0x7C}, /* 'u' */
    {0x1C,0x20,0x40,0x20,0x1C}, /* 'v' */
```

```
    {0x3C,0x40,0x30,0x40,0x3C}, /* 'w' */
    {0x44,0x28,0x10,0x28,0x44}, /* 'x' */
    {0x0C,0x50,0x50,0x50,0x3C}, /* 'y' */
    {0x44,0x64,0x54,0x4C,0x44}, /* 'z' */
    {0x00,0x08,0x36,0x41,0x00}, /* '{' */
    {0x00,0x00,0x7F,0x00,0x00}, /* '|' */
    {0x00,0x41,0x36,0x08,0x00}, /* '}' */
    {0x08,0x08,0x2A,0x1C,0x08}, /* '~' */
};

/*
 * oled_char_at: write one 5×7 character (6 px wide with 1 px gap).
 * Returns next column position.
 */
static uint8_t oled_ch(uint8_t page, uint8_t col, char c)
{
    uint8_t i;
    if (c < 0x20 || c > 0x7E) c = ' ';
    oled_pos(page, col);
    for (i = 0; i < 5; i++)
        oled_dat(pgm_read_byte(&FONT[c - 0x20][i]));
    oled_dat(0x00);
    return (uint8_t)(col + 6);
}

static void oled_str(uint8_t page, uint8_t col, const char *s)
{
    while (*s && col < SCR_W)
        col = oled_ch(page, col, *s++);
}

/*
 * oled_str_slide: draw string starting from x_offset (can be > 128 for off-screen).
 * Used for slide-in animation — caller passes decreasing x_offset each frame.
 * Characters outside 0..127 are simply skipped.
 */
static void oled_str_slide(uint8_t page, int16_t x_off, const char *s)
{
    int16_t col = x_off;
    while (*s) {
        if (col >= SCR_W) break;
        if (col >= 0) oled_ch(page, (uint8_t)col, *s);
        col += 6;
        s++;
    }
}

/* ============================================================================
 *  §5  NUMBER FORMATTING
 * ============================================================================ */
```

```c
/* Convert ×10 fixed-point to "XX.X" null-terminated string. */
static void fmt1(char *buf, int16_t v)
{
    uint8_t i = 0;
    if (v < 0) { buf[i++] = '-'; v = (int16_t)(-v); }
    uint16_t w = (uint16_t)(v / 10);
    uint8_t  f = (uint8_t)(v % 10);
    if (w >= 100) buf[i++] = (char)('0' + w / 100);
    if (w >=  10) buf[i++] = (char)('0' + (w / 10) % 10);
    buf[i++] = (char)('0' + w % 10);
    buf[i++] = '.';
    buf[i++] = (char)('0' + f);
    buf[i]   = '\0';
}

/* ============================================================================
 *  §6  FAN  —  Timer1 Fast-PWM 8-bit, OC1A = PB1 = D9
 *  f_PWM = 16 MHz / (8 × 256) ≈ 7.8 kHz
 * ============================================================================ */

#define FAN_OFF   0     /*  0 % */
#define FAN_MED   102   /* 40 % */
#define FAN_HIGH  230   /* 90 % */

static void fan_init(void)
{
    DDRB   |=  (1 << PB1);
    TCCR1A  = (1 << COM1A1) | (1 << WGM10);
    TCCR1B  = (1 << WGM12)  | (1 << CS11);
    OCR1A   = FAN_OFF;
}

/* ============================================================================
 *  §7  BUZZER  —  D8 / PB0, Active-LOW
 * ============================================================================ */

static void buz_init(void)
{
    DDRB  |=  (1 << PB0);
    PORTB |=  (1 << PB0);   /* HIGH = OFF */
}

static inline void buz_on(void)  { PORTB &= ~(1 << PB0); }
static inline void buz_off(void) { PORTB |=  (1 << PB0); }

/* Boot beep — 3 short + 1 long; called once before main loop. */
static void buz_boot(void)
{
    uint8_t i;
```

```
    for (i = 0; i < 3; i++) {
        buz_on();  _delay_ms(100);
        buz_off(); _delay_ms(70);
    }
    _delay_ms(120);
    buz_on();  _delay_ms(250);
    buz_off();
}

/* ==========================================================================
 *  §8  STATUS LEDs
 *  LED_GRN = D4/PD4   LED_YLW = D5/PD5   LED_RED = D6/PD6
 * ========================================================================== */

#define LED_G PD4
#define LED_Y PD5
#define LED_R PD6

static void leds_init(void)
{
    DDRD  |=  (1 << LED_G) | (1 << LED_Y) | (1 << LED_R);
    PORTD &= ~((1 << LED_G) | (1 << LED_Y) | (1 << LED_R));
}

static inline void led_g_on(void)  { PORTD |=  (1 << LED_G); }
static inline void led_g_off(void) { PORTD &= ~(1 << LED_G); }
static inline void led_y_on(void)  { PORTD |=  (1 << LED_Y); }
static inline void led_y_off(void) { PORTD &= ~(1 << LED_Y); }
static inline void led_r_on(void)  { PORTD |=  (1 << LED_R); }
static inline void led_r_off(void) { PORTD &= ~(1 << LED_R); }
static void leds_all_off(void)     { led_g_off(); led_y_off(); led_r_off(); }

/* ==========================================================================
 *  §9  BUTTONS  —  D2=RIGHT, D3=LEFT, D7=MAIN  (active LOW, pull-ups)
 * ========================================================================== */

#define BTN_R PD2
#define BTN_L PD3
#define BTN_M PD7

static void btn_init(void)
{
    DDRD  &= ~((1 << BTN_R) | (1 << BTN_L) | (1 << BTN_M));
    PORTD |=  ((1 << BTN_R) | (1 << BTN_L) | (1 << BTN_M));
}

/* 30 ms debounce — returns 1 if confirmed pressed */
static uint8_t btn_right(void)
{ if(!(PIND&(1<<BTN_R))){ _delay_ms(30); return !(PIND&(1<<BTN_R)); } return 0; }
```

```c
static uint8_t btn_left(void)
{ if(!(PIND&(1<<BTN_L))){ _delay_ms(30); return !(PIND&(1<<BTN_L)); } return 0; }

static uint8_t btn_main(void)
{ if(!(PIND&(1<<BTN_M))){ _delay_ms(30); return !(PIND&(1<<BTN_M)); } return 0; }

/* Block until release + 40 ms settle — prevents ghost presses */
static void rel_right(void){ while(!(PIND&(1<<BTN_R))); _delay_ms(40); }
static void rel_left(void) { while(!(PIND&(1<<BTN_L))); _delay_ms(40); }
static void rel_main(void) { while(!(PIND&(1<<BTN_M))); _delay_ms(40); }

/* ============================================================================
 *  §10  ALERT FSM  (hysteresis thresholds in hum_pct10 units)
 * ============================================================================ */

typedef enum { ST_NORMAL=0, ST_MEDIUM=1, ST_HIGH=2 } State;

/* Entry thresholds (going UP) */
#define HUM_MED_HI   310    /* 31.0 % — enter MEDIUM  */
#define HUM_HIGH_HI  710    /* 71.0 % — enter HIGH    */
/* Exit  thresholds (going DOWN — hysteresis gap ~1.5 %) */
#define HUM_MED_LO   285    /* 28.5 % — drop to NORMAL */
#define HUM_HIGH_LO  685    /* 68.5 % — drop to MEDIUM */

static State fsm_next(State cur, uint16_t h10)
{
    switch (cur) {
        case ST_NORMAL:
            if (h10 >= HUM_HIGH_HI) return ST_HIGH;
            if (h10 >= HUM_MED_HI)  return ST_MEDIUM;
            break;
        case ST_MEDIUM:
            if (h10 >= HUM_HIGH_HI) return ST_HIGH;
            if (h10 <= HUM_MED_LO)  return ST_NORMAL;
            break;
        case ST_HIGH:
            if (h10 <= HUM_MED_LO)  return ST_NORMAL;
            if (h10 <= HUM_HIGH_LO) return ST_MEDIUM;
            break;
    }
    return cur;
}

/* ============================================================================
 *  §11  OUTPUT CONTROLLER  —  LEDs + Fan + Buzzer, tick-synchronised
 *
 *  Called every 100 ms.  Contains ZERO blocking delays.
 *
 *  NORMAL : GRN steady ON  | Fan OFF   | Buzzer OFF
 *  MEDIUM : GRN+YLW ON     | Fan 40 %  | Buzzer 300 ms ON / 2700 ms OFF
```

```
 *  HIGH   : GRN+YLW+RED ON | Fan 90 %  | Buzzer continuous ON
 * ============================================================================ */

static void outputs_update(State st, uint8_t tick)
{
    switch (st) {

        case ST_NORMAL:
            OCR1A = FAN_OFF;
            buz_off();
            led_g_on();
            led_y_off();
            led_r_off();
            break;

        case ST_MEDIUM:
            OCR1A = FAN_MED;
            /* 3-second cycle: ON 300 ms (3 ticks), OFF 2700 ms (27 ticks) */
            if ((tick % 30) < 3) buz_on(); else buz_off();
            led_g_on();
            led_y_on();
            led_r_off();
            break;

        case ST_HIGH:
        default:
            OCR1A = FAN_HIGH;
            buz_on();          /* continuous */
            led_g_on();
            led_y_on();
            led_r_on();
            break;
    }
}

/* ============================================================================
 *  §12  GRAPH RING BUFFER
 * ============================================================================ */

#define GLEN 21

static int16_t  gt[GLEN];   /* temperature history */
static uint16_t gh[GLEN];   /* humidity    history */
static uint8_t  gi = 0;     /* next write index    */
static uint8_t  gc = 0;     /* count of valid entries */

static void gpush(int16_t t, uint16_t h)
{
    gt[gi] = t; gh[gi] = h;
    gi = (uint8_t)((gi + 1) % GLEN);
```

```
    if (gc < GLEN) gc++;
}

/* Map value to pixel row 0–39 (0=top, 39=bottom) */
static uint8_t gmap(int32_t v, int32_t lo, int32_t hi)
{
    if (v <= lo) return 39;
    if (v >= hi) return  0;
    return (uint8_t)(39L - ((v - lo) * 39L / (hi - lo)));
}

/*
 * gbar: draw a filled bar from top_row to row 39 at column col.
 * Uses page-level bitmask accumulation to avoid the pixel-overwrite bug
 * (multiple rows in the same OLED page-band must be ORed into one byte).
 */
static void gbar(uint8_t col, uint8_t top_row)
{
    uint8_t pg;
    for (pg = 0; pg < 5; pg++) {
        uint8_t pfirst = (uint8_t)(pg * 8);
        if (top_row > pfirst + 7) continue;
        uint8_t mask = 0x00, r;
        for (r = 0; r < 8; r++) {
            uint8_t pr = pfirst + r;
            if (pr >= top_row && pr <= 39)
                mask |= (uint8_t)(0x80 >> r);
        }
        if (mask) { oled_pos((uint8_t)(2 + pg), col); oled_dat(mask); }
    }
}

/* Single dot at (col, row) within the 40-px graph area */
static void gdot(uint8_t col, uint8_t row)
{
    if (row > 39) return;
    oled_pos((uint8_t)(2 + row / 8), col);
    oled_dat((uint8_t)(0x80 >> (row % 8)));
}

/* ============================================================================
 *  §13  SLIDE-IN SPLASH ANIMATIONS
 *
 *  Each frame the text column offset decreases from SCR_W → target_col.
 *  Step size = 8 px per frame, 30 ms per frame → ≈ 16 frames × 30 ms ≈ 480 ms.
 *  Any button press skips the animation immediately.
 * ============================================================================ */

#define SLIDE_STEP  8
#define SLIDE_MS    30
```

```c
/* Returns 1 if a button was pressed (skip request) */
static uint8_t any_btn(void)
{
    return (!(PIND & (1 << BTN_R)) ||
            !(PIND & (1 << BTN_L)) ||
            !(PIND & (1 << BTN_M)));
}

/* Wait ms milliseconds, returns early if button pressed */
static uint8_t wait_ms_check(uint16_t ms)
{
    uint16_t i;
    for (i = 0; i < ms; i++) {
        _delay_ms(1);
        if (any_btn()) return 1;
    }
    return 0;
}

/*
 * slide_in_line: animate one text line sliding in from the right.
 * page       — OLED page (0–7)
 * target_col — final resting column
 * text       — string to display
 * Returns 1 if button pressed.
 */
static uint8_t slide_in_line(uint8_t page, uint8_t target_col, const char *text)
{
    int16_t x = (int16_t)SCR_W;
    while (x > (int16_t)target_col) {
        oled_clr_row(page);
        oled_str_slide(page, x, text);
        if (any_btn()) {
            /* snap to final position and return */
            oled_clr_row(page);
            oled_str(page, target_col, text);
            return 1;
        }
        _delay_ms(SLIDE_MS);
        x -= SLIDE_STEP;
    }
    oled_clr_row(page);
    oled_str(page, target_col, text);
    return 0;
}

/* Hold a splash frame for `ms` ms, skippable by any button. */
static void splash_hold(uint16_t ms)
{
```

```c
    wait_ms_check(ms);
    /* flush release */
    _delay_ms(50);
}

/* ---------------------------------------------------------------------------
 *  Splash frame 1: Project name
 * --------------------------------------------------------------------------- */
static void splash_project(void)
{
    oled_clear();
    oled_row(0, 0xFF);
    oled_str(0, 14, "  SYSTEM  319  ");
    oled_row(1, 0xFF);

    if (slide_in_line(3, 10, "Smart Temp &")) return;
    if (slide_in_line(4,  8, "Humidity Monitor")) return;
    if (slide_in_line(5,  8, "ATmega328P | I2C")) return;

    oled_row(7, 0xFF);
    oled_str(7, 14, "Any key to skip");
    splash_hold(2500);
}

/* ---------------------------------------------------------------------------
 *  Splash frame 2: Lecturer + Group Leader
 * --------------------------------------------------------------------------- */
static void splash_lecturer(void)
{
    oled_clear();
    oled_row(0, 0xFF);
    oled_str(0, 14, "   EDUCATORS   ");
    oled_row(1, 0x01);

    if (slide_in_line(2, 4, "Lecturer:")) return;
    if (slide_in_line(3, 4, "Miss Aimen Najeeb")) return;
    oled_row(4, 0x08);
    if (slide_in_line(5, 4, "Group Leader:")) return;
    if (slide_in_line(6, 4, "M. Taha  EE-23319")) return;

    oled_row(7, 0xFF);
    oled_str(7, 14, "Any key to skip");
    splash_hold(2500);
}

/* ---------------------------------------------------------------------------
 *  Splash frame 3: Group members
 * --------------------------------------------------------------------------- */
static void splash_members(void)
{
```

```c
    oled_clear();
    oled_row(0, 0xFF);
    oled_str(0, 14, " GROUP MEMBERS ");
    oled_row(1, 0xFF);

    if (slide_in_line(2, 4, "Members:")) return;
    if (slide_in_line(3, 4, "Ali    EE-23328")) return;
    if (slide_in_line(4, 4, "Ahmed  EE-23325")) return;
    if (slide_in_line(5, 4, "Farzam EE-23326")) return;

    oled_row(7, 0xFF);
    oled_str(7, 14, "Any key to skip");
    splash_hold(2500);
}

/* ---------------------------------------------------------------------------
 *  Splash frame 4: System 319 + "System Ready" status
 * --------------------------------------------------------------------------- */
static void splash_ready(void)
{
    oled_clear();
    oled_row(0, 0xFF);
    oled_str(0, 14, "  SYSTEM  319  ");
    oled_row(1, 0xFF);

    if (slide_in_line(3, 20, "System 319")) return;
    if (slide_in_line(4,  8, "Initializing...")) return;
    if (slide_in_line(5, 14, "Please wait")) return;

    oled_row(7, 0xFF);
    oled_str(7, 20, "Starting up...");
    splash_hold(1500);

    /* "Ready" confirmation frame */
    oled_clr_row(4); oled_clr_row(5);
    oled_str(4,  8, "System Ready!");
    oled_str(5, 14, "Monitoring ON");
    splash_hold(1200);
}

/* ===========================================================================
 *  §14  MAIN DISPLAY  (Screen 0)
 *
 *  Page 0 — ████ SYSTEM 319 ████  solid header
 *  Page 1 — separator
 *  Page 2 — Temp: XX.X C
 *  Page 3 — Hum:  XX.X %
 *  Page 4 — separator
 *  Page 5 — [state message]
 *  Page 6 — humidity bar meter
```

```c
 *  Page 7 — [<] LEFT Info   [>] Graphs
 * ========================================================================== */

static void draw_main(int16_t t10, uint16_t h10, State st)
{
    char buf[22]; uint8_t len, c;

    /* Header */
    oled_row(0, 0xFF);
    oled_str(0, 10, "  SYSTEM  319  ");

    /* Separator */
    oled_row(1, 0x01);

    /* Temperature */
    oled_clr_row(2);
    strcpy(buf, "Temp: ");
    fmt1(buf + 6, t10);
    len = (uint8_t)strlen(buf);
    buf[len++] = ' '; buf[len++] = 'C'; buf[len] = '\0';
    oled_str(2, 2, buf);

    /* Humidity */
    oled_clr_row(3);
    strcpy(buf, "Hum:  ");
    fmt1(buf + 6, (int16_t)h10);
    len = (uint8_t)strlen(buf);
    buf[len++] = ' '; buf[len++] = '%'; buf[len] = '\0';
    oled_str(3, 2, buf);

    /* Separator */
    oled_row(4, 0x40);

    /* Status */
    oled_clr_row(5);
    switch (st) {
        case ST_NORMAL: oled_str(5, 2, "[OK]  System Normal "); break;
        case ST_MEDIUM: oled_str(5, 2, "[! ]  Medium Alert  "); break;
        default:        oled_str(5, 2, "[!!]  HIGH ALERT!!  "); break;
    }

    /* Humidity bar meter */
    oled_clr_row(6);
    {
        uint8_t pct = (h10 > 1000u) ? 100u : (uint8_t)(h10 / 10u);
        uint8_t bw  = (uint8_t)((uint16_t)pct * 116u / 100u);
        oled_pos(6, 6);
        for (c = 0; c < 116u; c++) oled_dat(c < bw ? 0x3C : 0x04);
    }
```

```c
    /* Navigation */
    oled_row(7, 0x80);
    oled_str(7, 2, "[<]Info [M]Main[>]Graph");
}

/* ============================================================================
 *  §15  INFO / STATUS SCREEN  (Screen 1 — Left side)
 *
 *  Shows: current state, which LEDs are ON, fan %, buzzer status.
 *  Page 0 — header
 *  Page 1 — Current State: NORMAL / MEDIUM / HIGH
 *  Page 2 — separator
 *  Page 3 — LEDs: GRN [on/off] YLW [on/off] RED [on/off]
 *  Page 4 — Fan:  OFF / 40% / 90%
 *  Page 5 — Buzzer: OFF / Intermittent / ON
 *  Page 6 — Threshold info
 *  Page 7 — nav
 * ============================================================================ */

static void draw_info(State st, uint16_t h10)
{
    char buf[22]; uint8_t len;

    oled_row(0, 0xFF);
    oled_str(0, 10, "  SYSTEM INFO  ");
    oled_row(1, 0xFF);

    /* State */
    oled_clr_row(2);
    switch (st) {
        case ST_NORMAL: oled_str(2, 2, "State: NORMAL       "); break;
        case ST_MEDIUM: oled_str(2, 2, "State: MEDIUM ALERT "); break;
        default:        oled_str(2, 2, "State: HIGH ALERT!! "); break;
    }

    oled_row(3, 0x08);

    /* LEDs */
    oled_clr_row(4);
    switch (st) {
        case ST_NORMAL:
            oled_str(4, 2, "LED: GRN  ylw  red"); break;
        case ST_MEDIUM:
            oled_str(4, 2, "LED: GRN  YLW  red"); break;
        default:
            oled_str(4, 2, "LED: GRN  YLW  RED"); break;
    }

    /* Fan + Buzzer */
    oled_clr_row(5);
```

```c
    switch (st) {
        case ST_NORMAL:
            oled_str(5, 2, "Fan:OFF  Buz:OFF  "); break;
        case ST_MEDIUM:
            oled_str(5, 2, "Fan:40%  Buz:INT  "); break;
        default:
            oled_str(5, 2, "Fan:90%  Buz:ON   "); break;
    }

    /* Humidity reading + threshold note */
    oled_clr_row(6);
    strcpy(buf, "Hum:");
    fmt1(buf + 4, (int16_t)h10);
    len = (uint8_t)strlen(buf);
    buf[len++] = '%'; buf[len] = '\0';
    /* append threshold reference */
    if (len < 18) {
        switch (st) {
            case ST_NORMAL: strcat(buf, " (<30%)"); break;
            case ST_MEDIUM: strcat(buf, " (<70%)"); break;
            default:        strcat(buf, " (>70%)"); break;
        }
    }
    oled_str(6, 2, buf);

    oled_row(7, 0x80);
    oled_str(7, 2, "[<]Graph[M]Main[>]Main");
}

/* ============================================================================
 *  §16  GRAPH PAGES
 * ============================================================================ */

static void draw_graph_temp(void)
{
    uint8_t i, col, c, r;
    oled_clear();

    oled_row(0, 0xFF);
    oled_str(0, 8, " TEMP HISTORY(C)");

    oled_str(2, 0, "50");
    oled_str(4, 0, "25");
    oled_str(6, 0, " 0");

    /* Y-axis */
    for (r = 0; r < 40; r++) gdot(14, r);
    /* 25 °C dotted guide */
    for (c = 16; c < SCR_W; c += 5) gdot(c, 20);
```

```
    {
        uint8_t s = (gc < GLEN) ? 0 : gi;
        for (i = 0; i < gc; i++) {
            uint8_t di   = (uint8_t)((s + i) % GLEN);
            uint8_t prow = gmap(gt[di], 0, 500);
            col = (uint8_t)(16 + i * 5);
            if (col + 2 >= SCR_W) break;
            gbar(col, prow); gbar((uint8_t)(col+1), prow); gbar((uint8_t)(col+2), prow);
        }
    }

    oled_row(7, 0x80);
    {
        char b[18]; strcpy(b, "Now:");
        uint8_t last = gi ? (uint8_t)(gi - 1) : (GLEN - 1);
        fmt1(b + 4, gt[last]);
        uint8_t l = (uint8_t)strlen(b); b[l++] = 'C'; b[l] = '\0';
        oled_str(7, 2, b);
    }
    oled_str(7, 68, "[<]Info[>]Hum");
}

static void draw_graph_hum(void)
{
    uint8_t i, col, c, r;
    oled_clear();

    oled_row(0, 0xFF);
    oled_str(0, 8, " HUM  HISTORY(%)");

    oled_str(2, 0, "99");
    oled_str(4, 0, "50");
    oled_str(6, 0, " 0");

    for (r = 0; r < 40; r++) gdot(14, r);

    /* 30 % boundary (sparse dots) */
    { uint8_t r30 = gmap(300, 0, 1000); for(c=16;c<SCR_W;c+=8) gdot(c,r30); }
    /* 50 % midline */
    { uint8_t r50 = gmap(500, 0, 1000); for(c=16;c<SCR_W;c+=5) gdot(c,r50); }
    /* 70 % HIGH threshold (dense) */
    { uint8_t r70 = gmap(700, 0, 1000); for(c=16;c<SCR_W;c+=3) gdot(c,r70); }

    {
        uint8_t s = (gc < GLEN) ? 0 : gi;
        for (i = 0; i < gc; i++) {
            uint8_t di   = (uint8_t)((s + i) % GLEN);
            uint8_t prow = gmap(gh[di], 0, 1000);
            col = (uint8_t)(16 + i * 5);
            if (col + 2 >= SCR_W) break;
```

```c
            gbar(col, prow); gbar((uint8_t)(col+1), prow); gbar((uint8_t)(col+2), prow);
        }
    }

    oled_row(7, 0x80);
    {
        char b[18]; strcpy(b, "Now:");
        uint8_t last = gi ? (uint8_t)(gi - 1) : (GLEN - 1);
        fmt1(b + 4, (int16_t)gh[last]);
        uint8_t l = (uint8_t)strlen(b); b[l++] = '%'; b[l] = '\0';
        oled_str(7, 2, b);
    }
    oled_str(7, 62, "[<]Temp[>]Main");
}

/* ==========================================================================
 *  §17  APPLICATION MAIN
 *
 *  Screens:
 *    0 = MAIN  (System 319 live readings)
 *    1 = INFO  (state details, LEDs, fan, buzzer)
 *    2 = TEMP GRAPH
 *    3 = HUM  GRAPH
 *
 *  Button mapping (runtime):
 *    RIGHT (D2) → screen = (screen + 1) % 4
 *    LEFT  (D3) → screen = (screen == 0) ? 3 : screen - 1
 *    MAIN  (D7) → screen = 0  (always returns to System 319 main)
 * ========================================================================== */

int main(void)
{
    int16_t T    = 250;          /* 25.0 °C default */
    uint16_t H   = 400;          /* 40.0 % default  */
    State   st   = ST_NORMAL;
    uint8_t sc   = 0;            /* current screen  */
    uint8_t redraw = 1;
    uint8_t tick   = 0;          /* 100 ms counter, wraps at 200 */

    /* ---- Init peripherals ---- */
    fan_init();
    buz_init();
    leds_init();
    btn_init();
    i2c_init();
    oled_init();
    aht20_init();

    /* ---- Slide-in splash sequence ---- */
    splash_project();
```

```c
    splash_lecturer();
    splash_members();
    splash_ready();

    /* Flush any button presses from splash skipping */
    _delay_ms(100);

    /* Boot confirmation beep */
    buz_boot();

    /* ================================================================
     *  MAIN LOOP
     * ================================================================ */
    for (;;)
    {
        /* Step 1: Read sensor */
        aht20_read(&T, &H);

        /* Step 2: Update FSM */
        st = fsm_next(st, H);

        /* Step 3: Push to graph history */
        gpush(T, H);

        /* Step 4: Render screen */
        if (redraw) {
            oled_clear();
            redraw = 0;
        }
        switch (sc) {
            case 1:  draw_info(st, H);  break;
            case 2:  draw_graph_temp(); break;
            case 3:  draw_graph_hum();  break;
            default: draw_main(T, H, st); break;
        }

        /* Step 5: Poll sub-loop (20 × 100 ms ≈ 2 s between reads) */
        {
            uint8_t lp;
            for (lp = 0; lp < 20; lp++)
            {
                /* Drive outputs — fully tick-based, zero blocking */
                outputs_update(st, tick);
                tick++;
                if (tick >= 200) tick = 0;

                /* RIGHT → next screen */
                if (btn_right()) {
                    rel_right();
                    sc = (uint8_t)((sc + 1) % 4);
```

```
                    redraw = 1; break;
                }

                /* LEFT → previous screen */
                if (btn_left()) {
                    rel_left();
                    sc = (sc == 0) ? 3u : (uint8_t)(sc - 1);
                    redraw = 1; break;
                }

                /* MAIN → System 319 main screen */
                if (btn_main()) {
                    rel_main();
                    sc = 0;
                    redraw = 1; break;
                }

                _delay_ms(100);
            }
        }
    }

    return 0;
}
```

Skill(s) to be assessed	Extent of Achievement			
	50%	65%	80%	100%
Application of theoretical knowledge for analysis	Unable to relate and recall any theory	Partially relates and recalls the theory	Recalls theoretical knowledge and relates it to the task without major mistakes	Recalls theoretical knowledge and relates it to the task with no mistake
Updates in Resource identification and allocation, task identification and scheduling, risk assessment and management	No updates or revision	----------	----------	Reasonable updates and revision presented
Updates in Bill of Materials (BOM) and budgeting	No updates in BOM and budgeting	----------	----------	Updates incorporated in BOM and budgeting with final expense
Realisation of design, testing and validation	Unable to synthesize and also unable to test/validate	Able to synthesize partially and able to test/validate OR Able to synthesize but partially able to test/validate	Able to synthesize and test/validate without major mistakes	Able to synthesize and test/validate without any mistake

Skill(s) to be assessed	Extent of Achievement			
	50%	65%	80%	100%
Application of theoretical knowledge for analysis	Unable to relate and recall any theory	Partially relates and recalls the theory	Recalls theoretical knowledge and relates it to the task without major mistakes	Recalls theoretical knowledge and relates it to the task with no mistake
Updates in Resource identification and allocation, task identification and scheduling, risk assessment and management	No updates or revision	----------	----------	Reasonable updates and revision presented
Updates in Bill of Materials (BOM) and budgeting	No updates in BOM and budgeting	----------	----------	Updates incorporated in BOM and budgeting with final expense
Realisation of design, testing and validation	Unable to synthesize and also unable to test/validate	Able to synthesize partially and able to test/validate OR Able to synthesize but partially able to test/validate	Able to synthesize and test/validate without major mistakes	Able to synthesize and test/validate without any mistake

Skill(s) to be assessed	Extent of Achievement			
	50%	65%	80%	100%
Application of theoretical knowledge for analysis	Unable to relate and recall any theory	Partially relates and recalls the theory	Recalls theoretical knowledge and relates it to the task without major mistakes	Recalls theoretical knowledge and relates it to the task with no mistake
Updates in Resource identification and allocation, task identification and scheduling, risk assessment and management	No updates or revision	----------	----------	Reasonable updates and revision presented
Updates in Bill of Materials (BOM) and budgeting	No updates in BOM and budgeting	----------	----------	Updates incorporated in BOM and budgeting with final expense
Realisation of design, testing and validation	Unable to synthesize and also unable to test/validate	Able to synthesize partially and able to test/validate OR Able to synthesize but partially able to test/validate	Able to synthesize and test/validate without major mistakes	Able to synthesize and test/validate without any mistake

Skill(s) to be assessed	Extent of Achievement			
	50%	65%	80%	100%
Application of theoretical knowledge for analysis	Unable to relate and recall any theory	Partially relates and recalls the theory	Recalls theoretical knowledge and relates it to the task without major mistakes	Recalls theoretical knowledge and relates it to the task with no mistake
Updates in Resource identification and allocation, task identification and scheduling, risk assessment and management	No updates or revision	----------	----------	Reasonable updates and revision presented
Updates in Bill of Materials (BOM) and budgeting	No updates in BOM and budgeting	----------	----------	Updates incorporated in BOM and budgeting with final expense
Realisation of design, testing and validation	Unable to synthesize and also unable to test/validate	Able to synthesize partially and able to test/validate OR Able to synthesize but partially able to test/validate	Able to synthesize and test/validate without major mistakes	Able to synthesize and test/validate without any mistake